

第4章: React Native / Expo入門

第3章で学んだReactの考え方を、いよいよ**スマホアプリの画面**に応用します。この章では、React Nativeの基本部品（**View** / **Text** / **Image** など）、画面の配置を決める **Flexbox**、そして画面の切り替えを担う **Expo Router** を、完全初心者向けに解説します。

1. React Native のコア部品

第3章の最後で触れたとおり、スマホアプリにはブラウザが無いので、HTMLタグ（`<div>` , `<p>`）は使えません。代わりにReact Nativeが用意した**専用部品**（**コアコンポーネント**）を使います。

1.1 主要部品の対応表

役割	WebのHTML	React Native	ひとこと
まとめり・箱	<code><div></code>	<code><View></code>	レイアウトの基本。すべての土台
文字の表示	<code><p></code> , <code></code>	<code><Text></code>	文字は必ずこれで囲む （重要ルール）
画像	<code></code>	<code><Image></code>	画像の表示
入力欄	<code><input></code>	<code><TextInput></code>	キーボード入力を受け取る
押せるもの	<code><button></code>	<code><Pressable></code> / <code><Button></code>	タップ操作を受け取る
縦スクロール	（自動）	<code><ScrollView></code> / <code><FlatList></code>	スマホは画面が小さいので必須

最重要ルール — 文字は必ず `<Text>` で囲む: Webでは `<div>こんにちは</div>` のように文字を直接置けますが、React Nativeでは文字（テキスト）は必ず `<Text>` の中に入れなければエラーになります。「`<View>` の中に文字を直接書いてはいけない、`<Text>` で包む」と覚えてください。これは初心者が最初に必ずつまづくポイントです。

1.2 View と Text — 最小のアプリ

第1章で作った `my-books-app` の `app/(tabs)/index.tsx` を、次の内容に書き換えて試してみましょう（既存の内容を消して貼り付け）。

```
// react-native という部品集から View と Text を借りてくる
import { View, Text } from "react-native";

// 画面のコンポーネント。export default で「この画面を公開」する
export default function HomeScreen() {
  return (
    // <View> : 箱。style で見た目を指定 (後述のFlexbox)
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      { /* flex:1 → 画面いっぱい広がる / justifyContent:"center" → 縦方向に中央 /
alignItems:"center" → 横方向に中央 */ }
      <Text style={{ fontSize: 24, fontWeight: "bold" }}><img alt="book icon" data-bbox="625 255 645 270"/> 書籍管理アプリ</Text>
      { /* fontSize:24 → 文字サイズ24 / fontWeight:"bold" → 太字 / 文字は必ず<Text>で囲む */ }
    </View>
  );
}
```

`style={{ ... }}` の中カッコが2つあるのはなぜ？ 外側の `{ }` は「JSXの中にTypeScriptを書く」ための中カッコ、内側の `{ }` は「スタイルを表すオブジェクト（第2章のオブジェクト）」です。だから `style={{ fontSize: 24 }}` は「styleにオブジェクト `{fontSize: 24}` を渡す」という意味になります。最初は2重に見えて戸惑いますが、慣れれば自然です。

Webのstyleとの違い: Webの `font-size` はReact Nativeでは `fontSize`（キャメルケース）、値も `"24px"` ではなく数値の `24` です。区切りのハイフンを無くして次の単語を大文字にする「キャメルケース」で書く、と覚えてください。

2. Flexbox — 部品を画面に配置する仕組み

スマホ画面に部品を「中央寄せ」「横並び」「均等配置」する仕組みが **Flexbox**（フレックスボックス）です。React Nativeのレイアウトは基本すべてこのFlexboxで行います。

2.1 主要なプロパティ

```
<View
  style={{
    flexDirection: "row",           // 子要素を並べる方向。"row"=横並び / "column"=縦並び（既定）
    justifyContent: "space-between", // 「主軸方向」の配置。下の表参照
    alignItems: "center",          // 「交差軸方向」の配置。下の表参照
    gap: 8,                         // 子要素同士の間隔（隙間）を8に
  }}
>
  <Text>左</Text>
  <Text>中</Text>
  <Text>右</Text>
</View>
```

プロパティ	役割	よく使う値
<code>flexDirection</code>	子要素を並べる向き	<code>"row"</code> （横） / <code>"column"</code> （縦・既定）
<code>justifyContent</code>	並べる方向の配置	<code>"center"</code> （中央） / <code>"space-between"</code> （両端に寄せ間を空ける） / <code>"flex-start"</code> （先頭寄せ）
<code>alignItems</code>	直角方向の配置	<code>"center"</code> （中央） / <code>"stretch"</code> （伸ばす） / <code>"flex-start"</code>
<code>flex</code>	余白をどう分けるか	<code>1</code> （利用可能なスペースいっぱいに広がる）
<code>gap</code>	子要素同士の隙間	<code>8</code> , <code>12</code> など数値
<code>padding</code>	自分の内側の余白	<code>16</code> など数値
<code>margin</code>	自分の外側の余白	<code>8</code> など数値

justifyContent と **alignItems** がやさしい理由: この2つは **flexDirection** の向きによって「効く方向」が入れ替わります。 - **flexDirection: "column"** (縦並び・既定) のとき → **justifyContent** は縦方向、**alignItems** は横方向 - **flexDirection: "row"** (横並び) のとき → **justifyContent** は横方向、**alignItems** は縦方向 最初は混乱しますが、「**justifyContent** = 並べてる向きの配置」「**alignItems** = それと直角の配置」と覚え、実際に値を変えて画面の変化を見ながら慣れるのが一番です。

flex: 1 の意味: 「使える空間を全部もらって広がる」という指定です。一番外側の **<View>** に **flex: 1** を付けると、その箱が画面全体に広がります。複数の子に **flex: 1** を付けると、空間を等分します。

2.2 StyleSheet で書く (推奨)

スタイルを **style={{ }}** の形で直接書くと、コードが読みにくなります。React Nativeには **StyleSheet** という、スタイルを別にまとめて名前と呼ぶ仕組みがあります。

```
import { View, Text, StyleSheet } from "react-native"; // StyleSheet も借りる

export default function HomeScreen() {
  return (
    <View style={styles.container}>      {/* styles.container で下の定義を参照 */}
      <Text style={styles.title}>📖 書籍管理アプリ</Text>
    </View>
  );
}

// StyleSheet.create({ ... }) : スタイルをまとめて定義する。名前を付けて再利用できる
const styles = StyleSheet.create({
  container: {                          // container という名前のスタイル
    flex: 1,                            // 画面いっぱい
    justifyContent: "center",           // 縦中央
    alignItems: "center",              // 横中央
    backgroundColor: "#fff",           // 背景色を白に (#fff は白を表す色コード)
  },
  title: {                              // title という名前のスタイル
    fontSize: 24,
    fontWeight: "bold",
    color: "#1e293b",                  // 文字色 (#1e293b は濃いグレー)
  },
});
```

StyleSheet.create を使うメリット: ①画面の構造 (JSX) とスタイルが分離されて読みやすい ②同じスタイルを使い回せる ③タイプミスを早く発見できる。第9章では、これに加えて **NativeWind** (Tailwind風の書き方) も導入し、さらに楽にスタイリングします。

3. ユーザー操作を受け取る部品

3.1 `TextInput` — 文字入力

検索ボックスや、本のタイトルを入力するフォームに使用します。

```
import { useState } from "react";
import { View, TextInput, Text, StyleSheet } from "react-native";

export default function SearchBox() {
  const [keyword, setKeyword] = useState(""); // 入力文字をstateで管理（最初は空文字""）

  return (
    <View>
      <TextInput
        style={styles.input} // 見た目
        placeholder="タイトルや著者で検索..." // placeholder : 未入力時に薄く表示する案内
        value={keyword} // value : 表示する値。stateと結びつける
        onChangeText={(text) => setKeyword(text)} // onChangeText : 入力が変わるたびに呼ばれる。textは今の入力内容
      />
      <Text>入力中: {keyword}</Text> {/* 入力した文字がリアルタイムで表示される */}
    </View>
  );
}

const styles = StyleSheet.create({
  input: {
    borderWidth: 1, // 枠線の太さ1
    borderColor: "#e2e8f0", // 枠線の色（薄いグレー）
    borderRadius: 8, // 角の丸み8
    padding: 10, // 内側の余白
  },
});
```

value と onChangeText のセット: この2つを組み合わせると「stateが入力欄の中身を管理し、入力が変わるとstateも更新される」という双方向の結びつきができます。これを「制御コンポーネント（controlled component）」と呼びます。フォームの基本パターンなので、第7章でも使います。

3.2 `Pressable` — タップを受け取る

Pressable（プレッサブル＝押せるもの）は、あらゆる部品を「押せるボタン」にできる万能の部品です。

```
import { Pressable, Text, StyleSheet } from "react-native";

export default function SaveButton() {
  return (
    // onPress : 押されたときに実行する処理を渡す
    <Pressable style={styles.button} onPress={() => console.log("保存ボタンが押された")}>
      <Text style={styles.buttonText}>保存する</Text>    {/* ボタンの中身も<Text>で囲む */}
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: "#1e40af", // 背景色 (青)
    paddingVertical: 13,        // 上下の内側余白
    borderRadius: 10,           // 角丸
    alignItems: "center",       // 中の文字を横中央に
  },
  buttonText: {
    color: "#fff",              // 文字色 (白)
    fontWeight: "bold",
    fontSize: 15,
  },
});
```

Button ではなく **Pressable** を使う理由: React Nativeには標準の **<Button>** もありますが、見た目のカスタマイズがほとんどできません。デザインを自由にしたい実際のアプリでは **Pressable** が定番です。本書も以降は **Pressable** を使います。

4. リストを表示する — **ScrollView** と **FlatList**

スマホは画面が小さいので、たくさん本を表示するにはスクロールが必須です。方法は2つあります。

4.1 **ScrollView** — 少数の要素向け

```
import { ScrollView, Text } from "react-native";

export default function SimpleList() {
  return (
    // ScrollView : 中身が画面を超えたらスクロールできる箱
    <ScrollView>
      <Text>本A</Text>
      <Text>本B</Text>
      <Text>本C</Text>
      { /* ...たくさん並べてもスクロールできる */ }
    </ScrollView>
  );
}
```

ScrollView の注意点: **ScrollView** は中身を全部いっぺんに描画します。本が10冊程度なら問題ありませんが、何百件もあるとアプリが重くなります。大量データには次の **FlatList** を使います。

4.2 **FlatList** — 大量データ向け（本書の主役）

FlatList（フラットリスト）は「画面に見えている分だけ描画し、スクロールに応じて順次描画する」賢いリストです。本書の書籍一覧はこれで作ります。

```
import { FlatList, View, Text, StyleSheet } from "react-native";

// 表示する本のデータ（後の章ではSupabaseから取得するが、ここでは仮データ）
const books = [
  { id: "1", title: "リーダブルコード", author: "Dustin Boswell" },
  { id: "2", title: "達人プログラマー", author: "David Thomas" },
  { id: "3", title: "TypeScript入門", author: "鈴木 僚太" },
];

export default function BookList() {
  return (
    <FlatList
      data={books} // data : 表示するデータの配列を渡す
      keyExtractor={(item) => item.id} // keyExtractor : 各要素を区別する一意のキーを返す (idを使う)
      renderItem={({ item }) => ( // renderItem : 1件分の見た目を返す関数。itemに1冊分が入る
        <View style={styles.card}>
          <Text style={styles.title}>{item.title}</Text> /* その本のタイトル */
          <Text style={styles.author}>{item.author}</Text> /* その本の著者 */
        </View>
      )}
    </>
  );
}

const styles = StyleSheet.create({
  card: { padding: 14, borderBottomWidth: 1, borderBottomColor: "#e2e8f0" },
  title: { fontSize: 15, fontWeight: "bold" },
  author: { fontSize: 12, color: "#64748b", marginTop: 3 },
});
```

FlatList の3つの必須プロパティ: - **data** ... 表示したいデータの配列。 - **renderItem** ... 「1件分をどう表示するか」を返す関数。 **{ item }** で1件ずつ受け取る。 - **keyExtractor** ... 各要素に固有の「目印 (key)」を付ける関数。Reactが要素を効率よく管理するために必要。本のidを使うのが定番です。

第3章で **map** を使ってカードを並べる話をしましたが、大量データでは **map** + **ScrollView** より **FlatList** の方が高速で、スマホアプリの定番です。

5. Expo Router — 画面の切り替え（ナビゲーション）

アプリには複数の画面があり、それらを行き来します（一覧→詳細→編集など）。この「画面遷移（ナビゲーション）」を担うのが **Expo Router**（エクスポ・ルーター）です。

5.1 ファイルを置くだけで画面ができる

Expo Routerの最大の特徴は「**app/** フォルダにファイルを置くと、それがそのまま1つの画面（ルート）になる」ことです。Web版チュートリアルのNext.jsと同じ「ファイルベースルーティング」の考え方です。

```
app/
├─ index.tsx      → アプリ起動時の最初の画面（URLでいう「/」）
├─ about.tsx      → /about でアクセスする画面
└─ books/
   └─ [id].tsx     → /books/1, /books/2 ... のような「動的な」画面
```

「ルーティング（routing）」とは？「どのURL（道筋）のとき、どの画面を表示するか」を決める仕組みのこと。「ルート（route）＝道筋」を割り当てるので routing です。Expo Routerでは、ファイルの場所がそのままルートになります。

[id].tsx の角カッコは何？ ファイル名を **[id].tsx** のように角カッコで囲むと、「どんな値でも受け取れる動的な画面」になります。**/books/1** でも **/books/99** でも、この1つのファイルが対応し、**id** の部分（1や99）を受け取れます。本の詳細・編集画面に使います（第6・8章）。

5.2 画面を移動する — **Link** と **useRouter**

別の画面へ移動する方法は2つあります。

```
// 方法1: <Link> - 押すと移動するリンク部品
import { Link } from "expo-router";
import { Text } from "react-native";

function Example1() {
  return (
    <Link href="/about">                                {/* href : 移動先のパス。押すと/about画面へ */}
      <Text>Aboutページへ</Text>
    </Link>
  );
}
```

```
// 方法2: useRouter - コードの中から移動する (ボタンを押した後など)
import { useRouter } from "expo-router";
import { Pressable, Text } from "react-native";

function Example2() {
  const router = useRouter(); // useRouter : 画面移動を操作する道具を取得

  return (
    <Pressable onPress={() => router.push("/books/1")}> /* router.push("パス") でその画面へ移動 */
      <Text>1冊目の詳細へ</Text>
    </Pressable>
  );
}
```

Link と router.push の使い分け: - **<Link>** ... 「押したら移動するリンク」を画面に置きたいとき。 - **router.push()** ... 「保存ボタンを押して、処理が終わったら一覧画面へ戻す」のように、コードの流れの中で移動したいとき。

router.push("/path") は新しい画面を上重ねる移動、**router.back()** は1つ前の画面に戻る移動です。第6章以降で実際に使います。

5.3 _layout.tsx — 画面全体の枠組み

app/ フォルダ内の **_layout.tsx** (アンダースコアで始まる特別なファイル) は、「全画面に共通する枠組み」を定義します。たとえば「画面上部のヘッダー」や「下部のタブバー」をここで設定します。

```
import { Stack } from "expo-router";

// _layout.tsx : このフォルダ内の画面たちを、どう積み重ねる(Stack)かを定義する
export default function RootLayout() {
  // Stack : 画面を「カードを重ねるように」遷移させるナビゲーションの形
  return <Stack />;
}
```

「**Stack (スタック)**」ナビゲーションとは？ 画面を「トランプのカードを上重ねていく」ように遷移させる方式です。一覧画面の上に詳細画面を重ね、「戻る」で上のカードをめくって一覧に戻る、というイメージ。スマホアプリで最も基本的な遷移方式です。第6章でこのStackと、下タブの「Tabs」ナビゲーションを実際に組み立てます。

6. 安全な表示領域 — SafeAreaView

最近のスマホは画面上部にノッチ（カメラ部分の切り欠き）や、下部にホームバーがあります。これらに文字が隠れないようにする部品が **SafeAreaView** です。

```
import { SafeAreaView } from "react-native-safe-area-context"; // この部品集から借りる
import { Text } from "react-native";

export default function Screen() {
  return (
    // SafeAreaView : ノッチやホームバーを避けた「安全な領域」に中身を表示する箱
    <SafeAreaView style={{ flex: 1 }}>
      <Text>この文字はノッチに隠れません</Text>
    </SafeAreaView>
  );
}
```

なぜ必要？ 普通の **<View>** だと、文字がカメラのノッチやステータスバーに重なって読めなくなることがあります。

SafeAreaView で囲むと、機種ごとの「危険地帯」を自動で避けてくれます。Expoのテンプレートには最初から組み込まれていることが多いです。

7. この章のまとめ

- React Nativeでは **View**（箱）・**Text**（文字、必ずこれで囲む）・**Image**・**TextInput**・**Pressable**などの専用部品を使う
- レイアウトは **Flexbox**（**flexDirection** / **justifyContent** / **alignItems** / **flex**）で決める
- スタイルは **StyleSheet.create** でまとめて管理する（第9章でNativeWindも導入）
- 大量のリストは **FlatList**（**data** / **renderItem** / **keyExtractor**）で表示する
- 画面遷移は **Expo Router**：**app/** にファイルを置けば画面になり、**<Link>** や **router.push()** で移動、**_layout.tsx** で全体の枠組みを定義
- **SafeAreaView** でノッチやホームバーを避ける

次の章へ: 画面の作り方が分かりました。第5章では、本のデータを保存する場所 = **Supabase**（データベース）をセットアップします。DBの選択肢の比較も詳しく解説します。